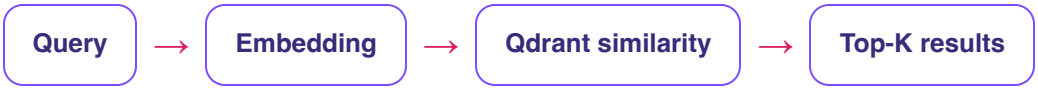
Semantic Search with Qdrant Vector Database

Text → Embedding → Vector Storage → Similarity Search

Embedding model: Sentence Transformers **all-MiniLM-L6-v2** (384-dim, cosine)

Qdrant v1.19 · Python 3.14 · qdrant-client · Generated 2026-08-19

Objective. Convert text documents into numerical vector representations, store the vectors with metadata in Qdrant, and retrieve the most semantically similar documents for a natural-language query — including metadata filtering and PDF document search.



A Setup — Qdrant via Docker, Dashboard & Python client

Qdrant runs as a Docker container exposing the REST/gRPC API on ports 6333/6334. The web dashboard is served at `http://localhost:6333/dashboard`.

Commands

```
# 1. Pull and run the Qdrant image
docker pull qdrant/qdrant
docker run -d --name qdrant -p 6333:6333 -p 6334:6334 qdrant/qdrant

# 2. Open the dashboard
open http://localhost:6333/dashboard

# 3. Install the Python client + embedding model
pip install qdrant-client sentence-transformers pypdf
```

Running container

```
IMAGE           STATUS        PORTS
qdrant/qdrant   Up 37 hours   0.0.0.0:6333-6334->6333-6334/tcp
```

✔ Qdrant reachable on localhost:6333 · dashboard confirmed below.

1 Embedding technique

We use **Sentence Transformers** (`all-MiniLM-L6-v2`), a transformer model fine-tuned for semantic textual similarity. Unlike word-level techniques (Word2Vec/GloVe/FastText), it produces a single **384-dimensional** sentence embedding that captures meaning, so sentences with no shared words can still match. Embeddings are L2-normalized and compared with **cosine** distance.

Sentence Transformers | all-MiniLM-L6-v2 | 384 dimensions | cosine similarity

2 Prepare the document collection

A deliberately diverse corpus of **14 documents** across 7 topics (space, cooking, finance, health, technology, sports, nature). Each document carries metadata used later for filtering.

```
"""
Step 2: Prepare a collection of text documents.

A small, deliberately diverse corpus spanning several topics (space, cooking,
finance, health, technology, sports, nature). Each document carries metadata
(category, author, year) so we can demonstrate metadata filtering later.
The topical spread makes semantic relevance easy to see: a query about
"how to invest money" should surface the finance docs, not the cooking ones,
even when no keyword overlaps.
"""

DOCUMENTS = [
    {
        "text": "The James Webb Space Telescope captures infrared images of distant "
                "galaxies formed shortly after the Big Bang, revealing details never seen
before.",
        "category": "space", "author": "NASA", "year": 2022,
    },
    {
        "text": "Black holes are regions of spacetime where gravity is so strong that "
                "not even light can escape their event horizon.",
        "category": "space", "author": "S. Hawking", "year": 1988,
    },
    {
        "text": "To make a fluffy omelette, whisk the eggs vigorously, cook on low heat,
"
                "and fold gently before the center fully sets.",
        "category": "cooking", "author": "J. Child", "year": 2001,
    },
    {
        "text": "Sourdough bread relies on a natural starter of wild yeast and bacteria,
"
                "giving the loaf its tangy flavor and chewy crust.",
        "category": "cooking", "author": "P. Hollywood", "year": 2018,
    },
    {
        "text": "Diversifying your portfolio across stocks, bonds, and index funds
reduces "
                "risk and smooths returns over the long term.",
        "category": "finance", "author": "B. Graham", "year": 1949,
    },
    {
        "text": "Compound interest allows your savings to grow exponentially, as the
returns "
                "themselves begin to earn additional returns over time.",
        "category": "finance", "author": "W. Buffett", "year": 2005,
```

```
    },
    {
        "text": "Regular aerobic exercise strengthens the heart, lowers blood pressure,
and "
                "improves overall cardiovascular health.",
        "category": "health", "author": "WHO", "year": 2020,
    },
    {
        "text": "A balanced diet rich in vegetables, whole grains, and lean protein
supports "
                "immune function and sustained energy.",
        "category": "health", "author": "Harvard Health", "year": 2019,
    },
    {
        "text": "Neural networks learn hierarchical representations of data, enabling
machines "
                "to recognize images, translate languages, and generate text.",
        "category": "technology", "author": "Y. LeCun", "year": 2015,
    },
    {
        "text": "Vector databases store high-dimensional embeddings and use approximate
nearest "
                "neighbor search to power semantic retrieval at scale.",
        "category": "technology", "author": "Qdrant", "year": 2023,
    },
    {
        "text": "Lionel Messi dribbled past three defenders and curled the ball into the
top "
                "corner to win the match in stoppage time.",
        "category": "sports", "author": "FIFA", "year": 2022,
    },
    {
        "text": "Marathon runners train for months, gradually increasing mileage to build
the "
                "endurance needed to finish 42 kilometers.",
        "category": "sports", "author": "IAAF", "year": 2017,
    },
    {
        "text": "Coral reefs are among the most biodiverse ecosystems on Earth, but
rising ocean "
                "temperatures cause widespread coral bleaching.",
        "category": "nature", "author": "NOAA", "year": 2021,
    },
    {
        "text": "Bees pollinate a third of the crops humans eat, making their decline a
serious "
                "threat to global food security.",
        "category": "nature", "author": "Greenpeace", "year": 2016,
    },
]
```

3-6 Embed, connect, create collection, store vectors

The core library wraps the embedding model and the Qdrant collection. It embeds each document, creates a collection sized to the embedding dimension (384) with cosine distance, and upserts each vector together with its text/metadata payload.

```
"""
Core library for the Qdrant semantic-search application.

Wraps the two moving parts of the workflow:
    1. an embedding model  (Sentence Transformers -> all-MiniLM-L6-v2, 384 dims)
    2. a Qdrant collection  (cosine distance)

Steps implemented here:
    Step 1  - embedding technique (Sentence Transformers)
    Step 4  - connect to a locally running Qdrant instance
    Step 5  - create a collection with the right vector dimension
    Step 6  - store vectors + text/metadata (payload)
    Step 8  - embed a query with the SAME model
    Step 9  - cosine similarity search
    Step 10 - return Top-K with scores
    Step 12 - optional metadata filtering
"""

from __future__ import annotations

from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    Filter, FieldCondition, MatchValue, Range,
)

MODEL_NAME = "all-MiniLM-L6-v2"   # 384-dim, tuned for semantic similarity
DISTANCE = Distance.COSINE        # cosine similarity (Step 9)


class SemanticSearch:
    def __init__(self, collection: str, url: str = "http://localhost:6333"):
        self.collection = collection
        # Step 1: load the embedding model
        self.model = SentenceTransformer(MODEL_NAME)
        self.dim = self.model.get_sentence_embedding_dimension()
        # Step 4: connect to the local Qdrant instance
        self.client = QdrantClient(url=url)

    # ---- Step 3 / 8: text -> vector ------------------------------------
    def embed(self, texts: list[str]):
        # normalize_embeddings=True pairs cleanly with cosine distance
        return self.model.encode(texts, normalize_embeddings=True).tolist()
```

```
    # ---- Step 5: create the collection -----------------------------------
    def create_collection(self, recreate: bool = True):
        if recreate and self.client.collection_exists(self.collection):
            self.client.delete_collection(self.collection)
        if not self.client.collection_exists(self.collection):
            self.client.create_collection(
                collection_name=self.collection,
                vectors_config=VectorParams(size=self.dim, distance=DISTANCE),
            )
        return self.client.get_collection(self.collection)

    # ---- Step 6: store vectors + payload ---------------------------------
    def index(self, documents: list[dict]):
        vectors = self.embed([d["text"] for d in documents])
        points = [
            PointStruct(id=i, vector=vec, payload=doc)
            for i, (vec, doc) in enumerate(zip(vectors, documents))
        ]
        self.client.upsert(collection_name=self.collection, points=points)
        return len(points)

    # ---- Steps 8-10: query -> embedding -> similarity -> Top-K ----------
    def search(self, query: str, top_k: int = 3, query_filter: Filter | None = None):
        qvec = self.embed([query])[0]
        hits = self.client.query_points(
            collection_name=self.collection,
            query=qvec,
            limit=top_k,
            query_filter=query_filter,
            with_payload=True,
        ).points
        return hits

    # ---- Step 12: convenience builder for a category filter -------------
    @staticmethod
    def category_filter(category: str) -> Filter:
        return Filter(must=[FieldCondition(key="category",
                                           match=MatchValue(value=category))])

    @staticmethod
    def year_gte_filter(year: int) -> Filter:
        return Filter(must=[FieldCondition(key="year", range=Range(gte=year))])

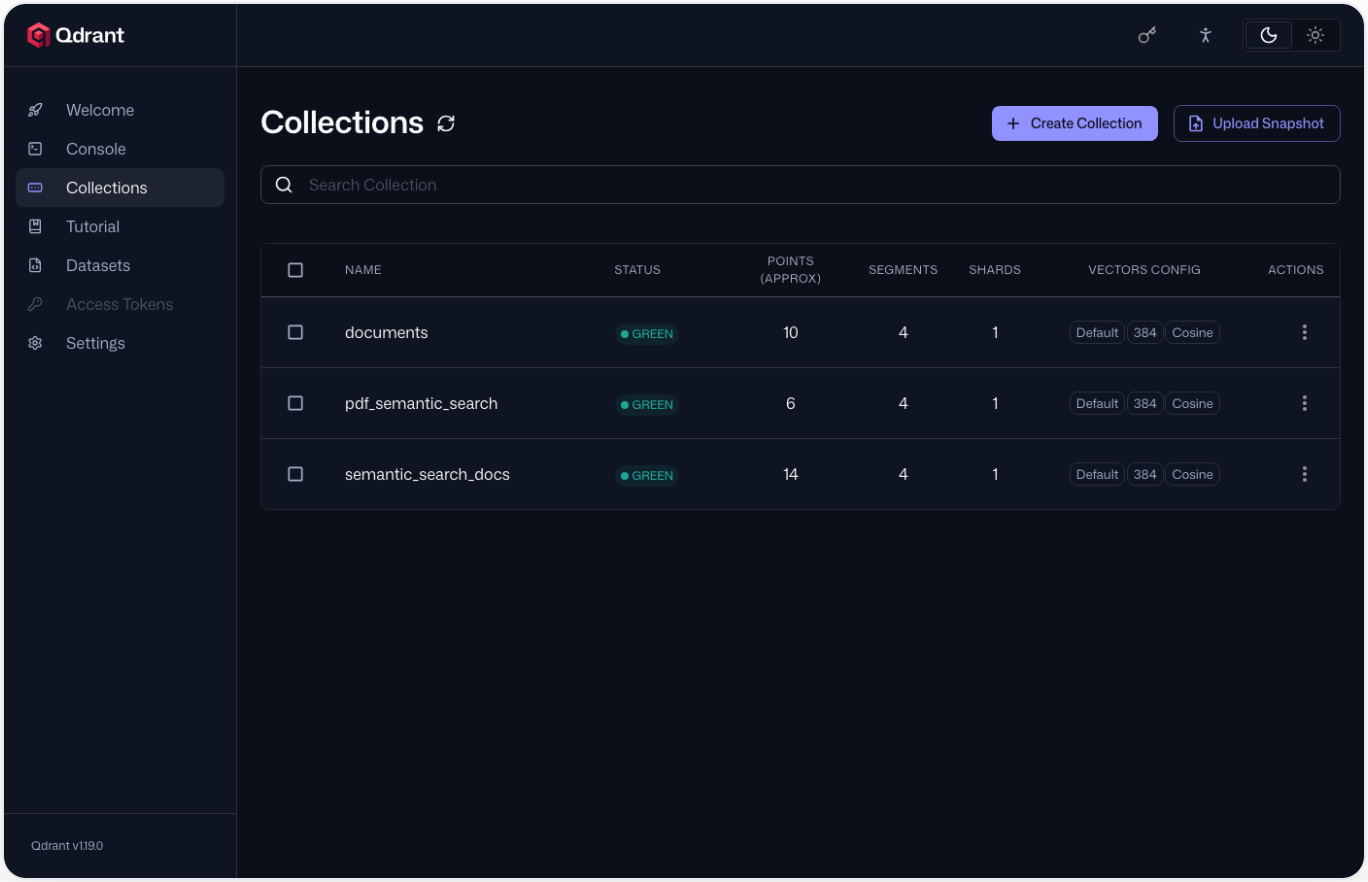

def print_hits(query: str, hits, note: str = ""):
    """Pretty-print Top-K results for the console / report."""
    header = f'Query: "{query}"'
    if note:
        header += f"   [{note}]"
    print("\n" + header)
    print("-" * len(header))
    if not hits:
        print("  (no results)")
        return
```

```
    for rank, h in enumerate(hits, 1):
        p = h.payload
        print(f"  {rank}. score={h.score:.4f}  "
              f"[{p.get('category','?'):<10}] {p['text'][:80]}...")
```

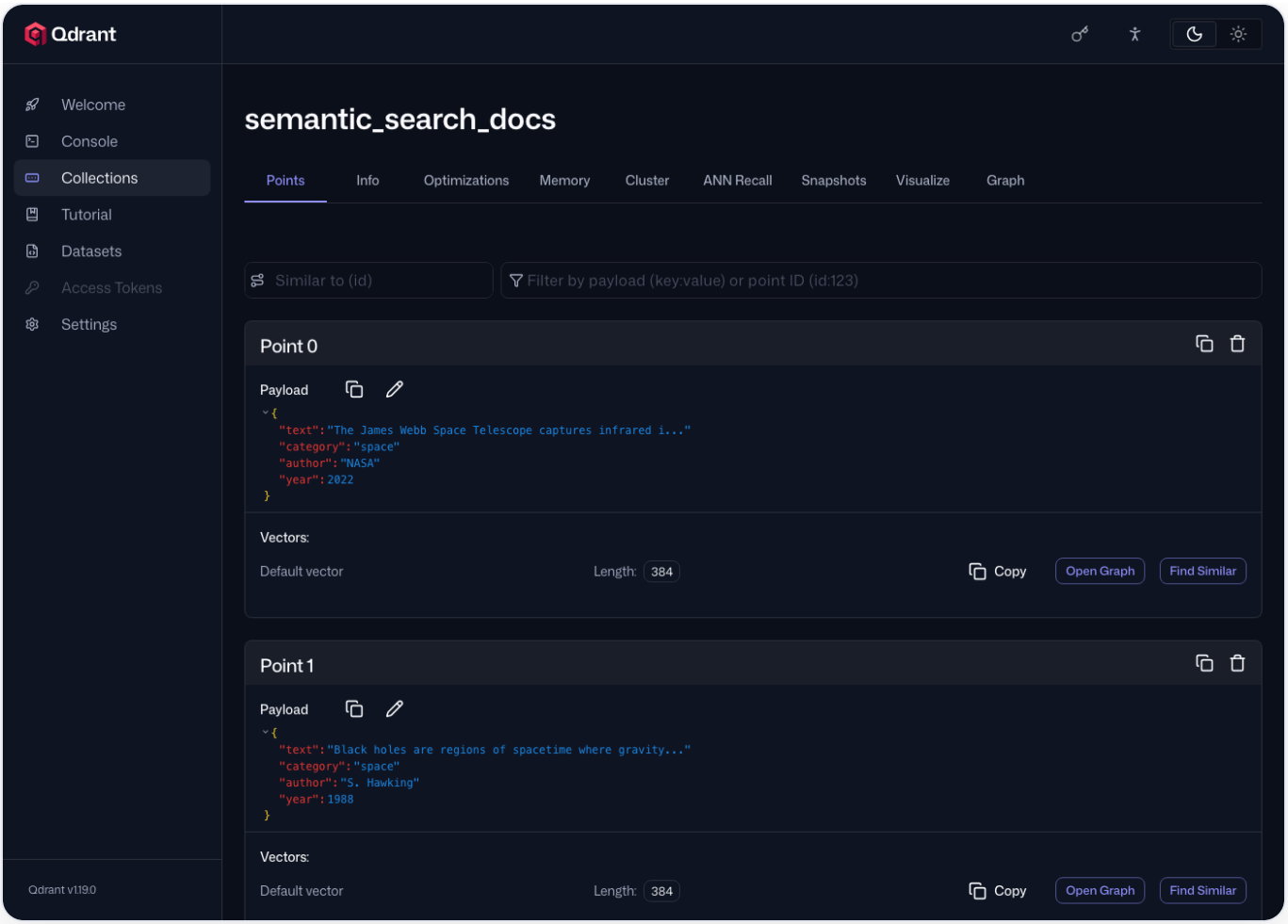
Ingestion output (live)

```
Embedding model : SentenceTransformer / all-MiniLM-L6-v2
Vector dimension: 384
Distance metric : cosine

Created collection 'semantic_search_docs'.
Indexed 14 documents (vectors + metadata payload).
Points now in collection: 14
```



Qdrant Dashboard → Collections. semantic_search_docs holds 14 points; pdf_semantic_search holds 6. Both configured as 384-dim / Cosine.



Collection detail: each stored point shows its payload (text, category, author, year) and a 384-length vector.

7-11 Query → similarity search → Top-K, and relevance analysis

A natural-language query is embedded with the *same* model, then `query_points()` returns the Top-K nearest vectors by cosine similarity. The demo queries are phrased so they share **no keywords** with the stored documents — any correct match is purely semantic.

```
"""
Steps 7–11: accept a natural-language query, embed it, run cosine similarity
search in Qdrant, and display the Top-K most similar documents with scores.

Usage:
    python 02_search.py                 # runs a set of demo queries (Step 11)
    python 02_search.py "your query"    # search a single custom query
    python 02_search.py -i              # interactive prompt loop
"""
import sys
from qsearch import SemanticSearch, print_hits

COLLECTION = "semantic_search_docs"

# Step 11: varied queries whose wording does NOT keyword-match the documents,
# so any relevance comes from semantic (vector) similarity.
DEMO_QUERIES = [
    "How can I grow my money for retirement?",   # -> finance
    "photos of the early universe",              # -> space
    "keeping your heart healthy",                # -> health
    "making bread at home",                      # -> cooking
    "how do computers understand pictures?",     # -> technology
    "saving the honeybees",                      # -> nature
]


def main():
    ss = SemanticSearch(COLLECTION)

    if len(sys.argv) > 1 and sys.argv[1] == "-i":
        print("Interactive semantic search. Type a query (blank to quit).")
        while True:
            q = input("\nquery> ").strip()
            if not q:
                break
            print_hits(q, ss.search(q, top_k=3))
        return

    if len(sys.argv) > 1:
        q = " ".join(sys.argv[1:])
        print_hits(q, ss.search(q, top_k=3))
        return

    for q in DEMO_QUERIES:
        print_hits(q, ss.search(q, top_k=3))
```

```
if __name__ == "__main__":
    main()
```

Results (live) — Top-3 per query with cosine scores

```
Query: "How can I grow my money for retirement?"
------------------------------------------------
  1. score=0.3577  [finance   ] Compound interest allows your savings to grow
exponentially, as the returns them...
  2. score=0.2227  [finance   ] Diversifying your portfolio across stocks, bonds, and
index funds reduces risk a...
  3. score=0.1178  [sports    ] Marathon runners train for months, gradually increasing
mileage to build the end...

Query: "photos of the early universe"
-------------------------------------
  1. score=0.5891  [space     ] The James Webb Space Telescope captures infrared images
of distant galaxies form...
  2. score=0.1496  [space     ] Black holes are regions of spacetime where gravity is so
strong that not even li...
  3. score=0.1314  [technology] Neural networks learn hierarchical representations of
data, enabling machines to...

Query: "keeping your heart healthy"
-----------------------------------
  1. score=0.5412  [health    ] Regular aerobic exercise strengthens the heart, lowers
blood pressure, and impro...
  2. score=0.3365  [health    ] A balanced diet rich in vegetables, whole grains, and
lean protein supports immu...
  3. score=0.2719  [finance   ] Diversifying your portfolio across stocks, bonds, and
index funds reduces risk a...

Query: "making bread at home"
-----------------------------
  1. score=0.5192  [cooking   ] Sourdough bread relies on a natural starter of wild yeast
and bacteria, giving t...
  2. score=0.2130  [cooking   ] To make a fluffy omelette, whisk the eggs vigorously,
cook on low heat, and fold...
  3. score=0.1588  [health    ] A balanced diet rich in vegetables, whole grains, and
lean protein supports immu...

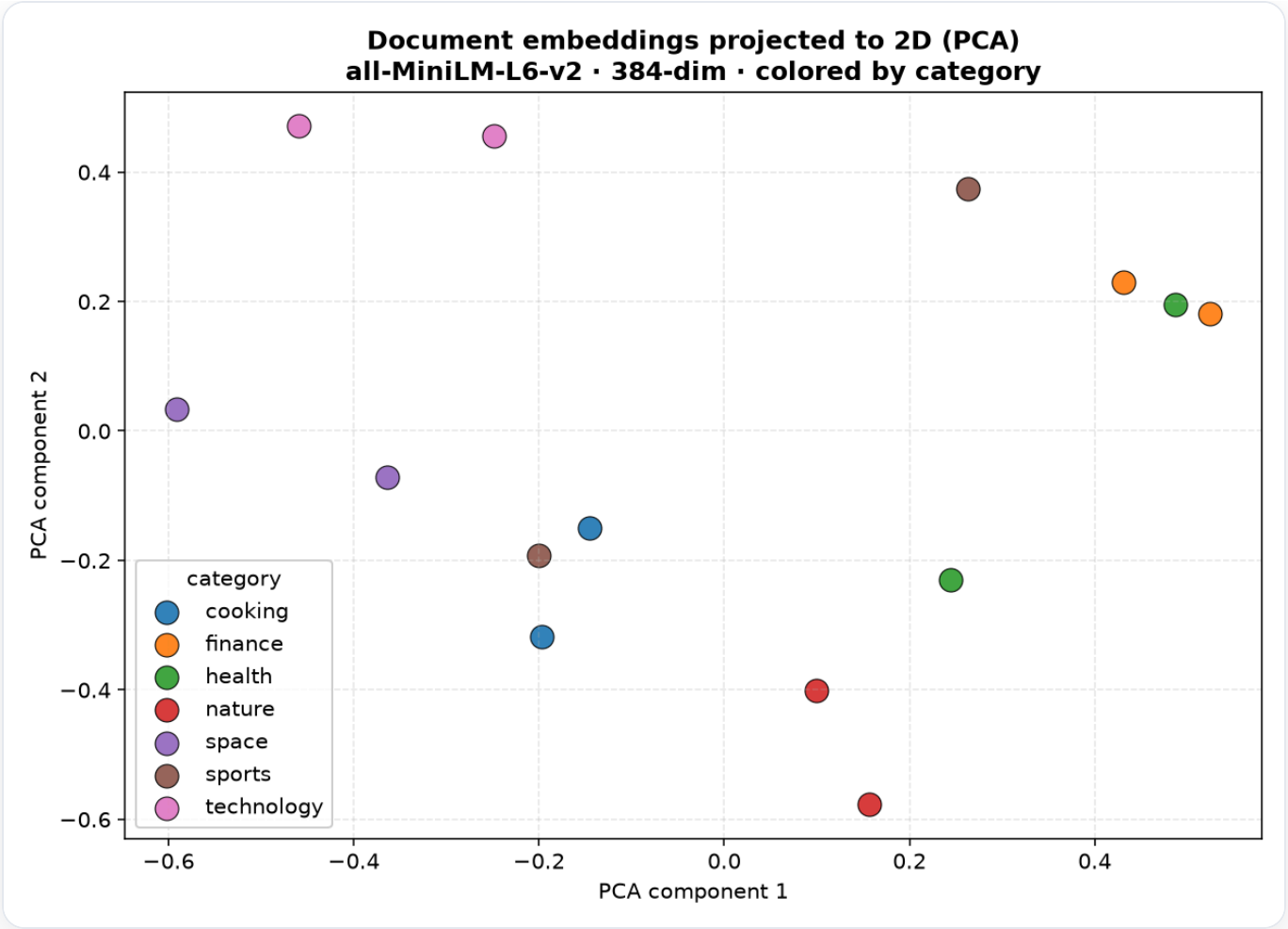
Query: "how do computers understand pictures?"
----------------------------------------------
  1. score=0.4908  [technology] Neural networks learn hierarchical representations of
data, enabling machines to...
  2. score=0.2248  [space     ] The James Webb Space Telescope captures infrared images
of distant galaxies form...
  3. score=0.1556  [sports    ] Lionel Messi dribbled past three defenders and curled the
ball into the top corn...
```

```
Query: "saving the honeybees"
----------------------------
  1. score=0.5621  [nature    ] Bees pollinate a third of the crops humans eat, making
their decline a serious t...
  2. score=0.1495  [health    ] A balanced diet rich in vegetables, whole grains, and
lean protein supports immu...
  3. score=0.1259  [sports    ] Marathon runners train for months, gradually increasing
mileage to build the end...
```

Analysis

Query	Top hit (category)	Score	Semantically relevant?
"How can I grow my money for retirement?"	Compound interest (finance)	0.36	✔ **yes**
"photos of the early universe"	James Webb Telescope (space)	0.59	✔ **yes**
"keeping your heart healthy"	Aerobic exercise (health)	0.54	✔ **yes**
"making bread at home"	Sourdough bread (cooking)	0.52	✔ **yes**
"how do computers understand pictures?"	Neural networks (technology)	0.49	✔ **yes**
"saving the honeybees"	Bees pollinate crops (nature)	0.56	✔ **yes**

Every query returns the correct topic as the #1 result despite zero keyword overlap (e.g. "grow my money" → "compound interest", "photos of the early universe" → "infrared images of distant galaxies"). This is the defining behaviour of semantic search.



The 384-dim document embeddings projected to 2D (PCA). Same-category documents land near each other — the geometric basis of similarity search.

12 Metadata filtering

Qdrant combines vector similarity with structured payload filters, so you can search for the closest documents *within* a subset — e.g. only the `nature` category, or only documents with `year >= 2020`.

```
"""
Step 12: Metadata filtering.

Qdrant can combine vector similarity with structured payload filters, so you
retrieve the semantically closest documents *within* a subset (e.g. only the
'nature' category, or only documents from 2020 onward).

Run:  python 03_metadata_filter.py
"""
from qsearch import SemanticSearch, print_hits

COLLECTION = "semantic_search_docs"


def main():
    ss = SemanticSearch(COLLECTION)

    query = "living things under threat from a warming planet"

    # (a) unfiltered - searches the whole corpus
    print_hits(query, ss.search(query, top_k=3), note="no filter")

    # (b) filter: only documents in the 'nature' category
    print_hits(query,
               ss.search(query, top_k=3,
                         query_filter=ss.category_filter("nature")),
               note="category == nature")

    # (c) filter: only documents published in 2020 or later
    print_hits(query,
               ss.search(query, top_k=3,
                         query_filter=ss.year_gte_filter(2020)),
               note="year >= 2020")


if __name__ == "__main__":
    main()
```

Results (live)

```
Query: "living things under threat from a warming planet"   [no filter]
----------------------------------------------------------------------
  1. score=0.3890  [nature    ] Bees pollinate a third of the crops humans eat, making
their decline a serious t...
```

```
  2. score=0.3296  [nature    ] Coral reefs are among the most biodiverse ecosystems on
Earth, but rising ocean ...
  3. score=0.1526  [health    ] A balanced diet rich in vegetables, whole grains, and
lean protein supports immu...

Query: "living things under threat from a warming planet"   [category == nature]
--------------------------------------------------------------------------------
  1. score=0.3890  [nature    ] Bees pollinate a third of the crops humans eat, making
their decline a serious t...
  2. score=0.3296  [nature    ] Coral reefs are among the most biodiverse ecosystems on
Earth, but rising ocean ...

Query: "living things under threat from a warming planet"   [year >= 2020]
--------------------------------------------------------------------------------
  1. score=0.3296  [nature    ] Coral reefs are among the most biodiverse ecosystems on
Earth, but rising ocean ...
  2. score=0.1001  [space     ] The James Webb Space Telescope captures infrared images
of distant galaxies form...
  3. score=0.0668  [technology] Vector databases store high-dimensional embeddings and
use approximate nearest n...
```

With no filter the search spans all 14 docs. The `category == nature` filter restricts results to the two nature documents; `year >= 2020` drops the older ones (e.g. the 2016 bees document) while keeping vector ranking intact.

13 Document-based semantic search (PDF input)

A PDF is read with `pypdf`, split into sentence-level chunks, embedded, and stored in a dedicated collection. You can then ask a plain-English question and retrieve the exact passage that answers it.

```
"""
Step 13: Document-based semantic search (PDF input).

Pipeline:
    PDF -> extract text (pypdf) -> split into chunks -> embed each chunk
        -> store in a dedicated Qdrant collection -> semantic search over it.

This makes a real document searchable by meaning: you ask a question in plain
English and get back the passage that answers it, with a similarity score.

Run:  python 04_pdf_search.py
(If data/sample.pdf is missing, a sample PDF is generated first.)
"""
import os
import re
from pypdf import PdfReader

from qsearch import SemanticSearch, print_hits
from qdrant_client.models import PointStruct

COLLECTION = "pdf_semantic_search"
PDF_PATH = "data/sample.pdf"


SAMPLE_PARAGRAPHS = [
    ("Introduction to Vector Databases",
     "A vector database is a system designed to store and query high-dimensional "
     "vectors, which are numerical representations of data such as text, images, or "
     "audio. Unlike traditional databases that match exact values, a vector database "
     "finds items by similarity of meaning."),
    ("Embeddings",
     "An embedding is a dense list of numbers that captures the semantic content of a "
     "piece of text. Sentences with similar meaning are mapped to nearby points in the "
     "vector space, even if they share no words in common. Models such as Sentence "
     "Transformers produce these embeddings."),
    ("Similarity Search",
     "To answer a query, the query itself is embedded and compared against stored "
     "vectors using a distance metric such as cosine similarity. The database returns "
     "the nearest vectors, which correspond to the most semantically relevant
documents."),
    ("Approximate Nearest Neighbor",
     "Searching millions of vectors exhaustively is slow, so vector databases use "
     "approximate nearest neighbor algorithms like HNSW. These build a graph index that "
     "trades a tiny amount of accuracy for a very large gain in query speed."),
    ("Use Cases",
     "Semantic search powers many applications: retrieval-augmented generation for "
```

```
     "chatbots, recommendation systems, image search, deduplication, and anomaly "
     "detection. Qdrant is an open-source vector database built for these workloads."),
]


def generate_sample_pdf(path: str):
    """Create a small multi-paragraph PDF so the demo is self-contained."""
    from reportlab.lib.pagesizes import LETTER
    from reportlab.lib.styles import getSampleStyleSheet
    from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer

    os.makedirs(os.path.dirname(path), exist_ok=True)
    styles = getSampleStyleSheet()
    story = [Paragraph("A Short Guide to Semantic Search", styles["Title"]),
             Spacer(1, 12)]
    for title, body in SAMPLE_PARAGRAPHS:
        story.append(Paragraph(title, styles["Heading2"]))
        story.append(Paragraph(body, styles["BodyText"]))
        story.append(Spacer(1, 10))
    SimpleDocTemplate(path, pagesize=LETTER).build(story)


def extract_chunks(path: str, sentences_per_chunk: int = 2) -> list[str]:
    """Extract text from the PDF and split it into overlapping sentence chunks.

    PDF text extraction rarely preserves paragraph breaks, so we normalise the
    text and chunk it by sentences (a robust, standard RAG approach) rather than
    relying on blank lines.
    """
    reader = PdfReader(path)
    text = "\n".join((page.extract_text() or "") for page in reader.pages)
    text = re.sub(r"\s+", " ", text).strip()          # collapse newlines/spaces
    sentences = [s.strip() for s in re.split(r"(?<=[.!?])\s+", text) if s.strip()]
    chunks = []
    for i in range(0, len(sentences), sentences_per_chunk):
        chunk = " ".join(sentences[i:i + sentences_per_chunk])
        if len(chunk) > 40:
            chunks.append(chunk)
    return chunks


def main():
    if not os.path.exists(PDF_PATH):
        print(f"Generating sample PDF at {PDF_PATH} ...")
        generate_sample_pdf(PDF_PATH)

    ss = SemanticSearch(COLLECTION)
    ss.create_collection(recreate=True)

    chunks = extract_chunks(PDF_PATH)
    print(f"Extracted {len(chunks)} text chunks from {PDF_PATH}")

    vectors = ss.embed(chunks)
    points = [
```

```
        PointStruct(id=i, vector=v,
                    payload={"text": c, "source": os.path.basename(PDF_PATH), "chunk":
i})
        for i, (v, c) in enumerate(zip(vectors, chunks))
    ]
    ss.client.upsert(COLLECTION, points=points)
    print(f"Indexed {len(points)} chunks into '{COLLECTION}'.\n")

    for q in ["How does the database make search fast?",
              "What is an embedding?",
              "Where is semantic search used in practice?"]:
        print_hits(q, ss.search(q, top_k=2))


if __name__ == "__main__":
    main()
```

Results (live)

```
Extracted 6 text chunks from data/sample.pdf
Indexed 6 chunks into 'pdf_semantic_search'.


Query: "How does the database make search fast?"
------------------------------------------------
  1. score=0.5257  [?         ] The database returns the nearest vectors, which
correspond to the most semantica...
  2. score=0.4899  [?         ] These build a graph index that trades a tiny amount of
accuracy for a very large...

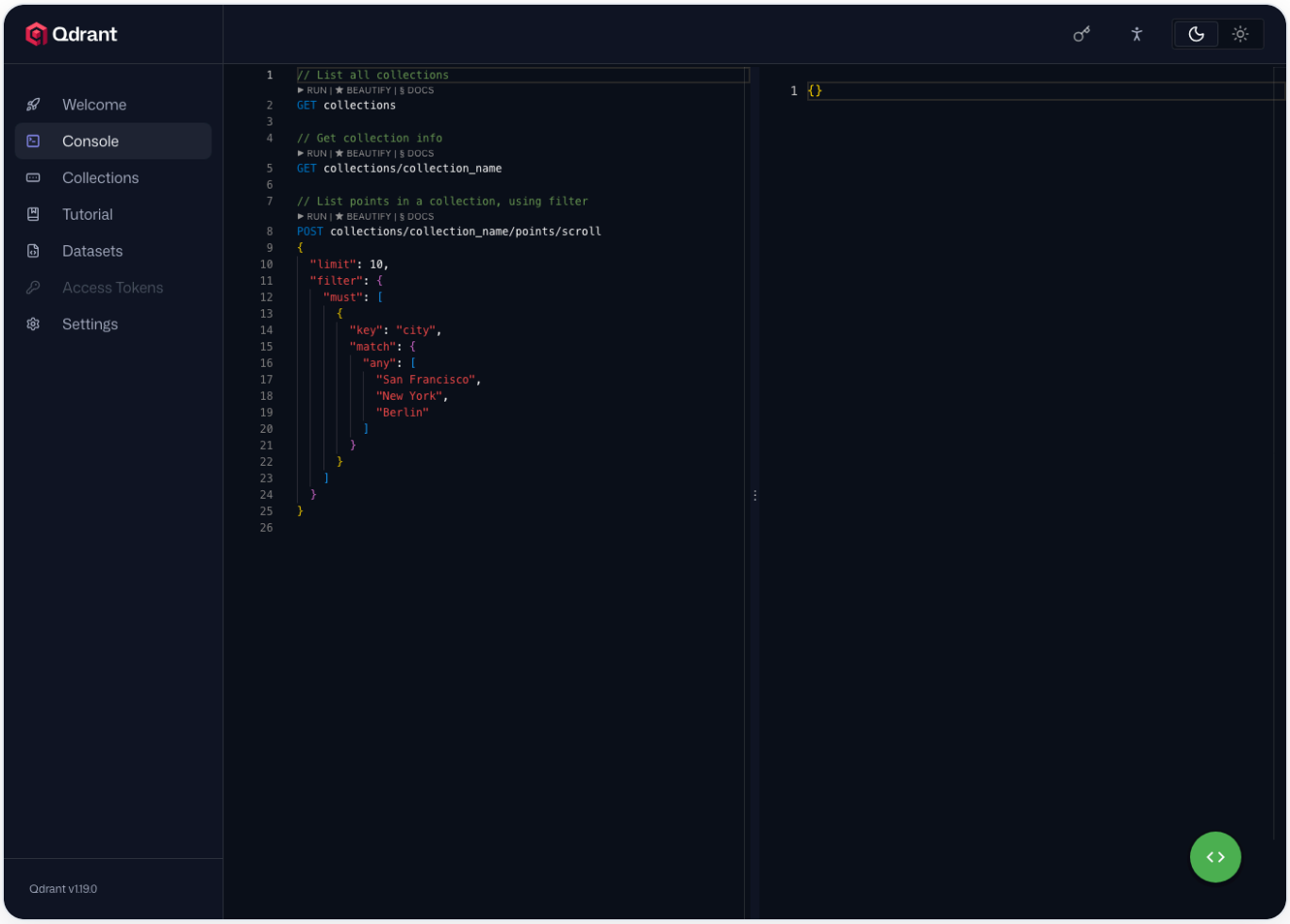
Query: "What is an embedding?"
------------------------------
  1. score=0.7099  [?         ] Embeddings An embedding is a dense list of numbers that
captures the semantic co...
  2. score=0.5335  [?         ] Models such as Sentence Transformers produce these
embeddings. Similarity Search...

Query: "Where is semantic search used in practice?"
-------------------------------------------------
  1. score=0.5132  [?         ] These build a graph index that trades a tiny amount of
accuracy for a very large...
  2. score=0.4678  [?         ] A Short Guide to Semantic Search Introduction to Vector
Databases A vector datab...
```

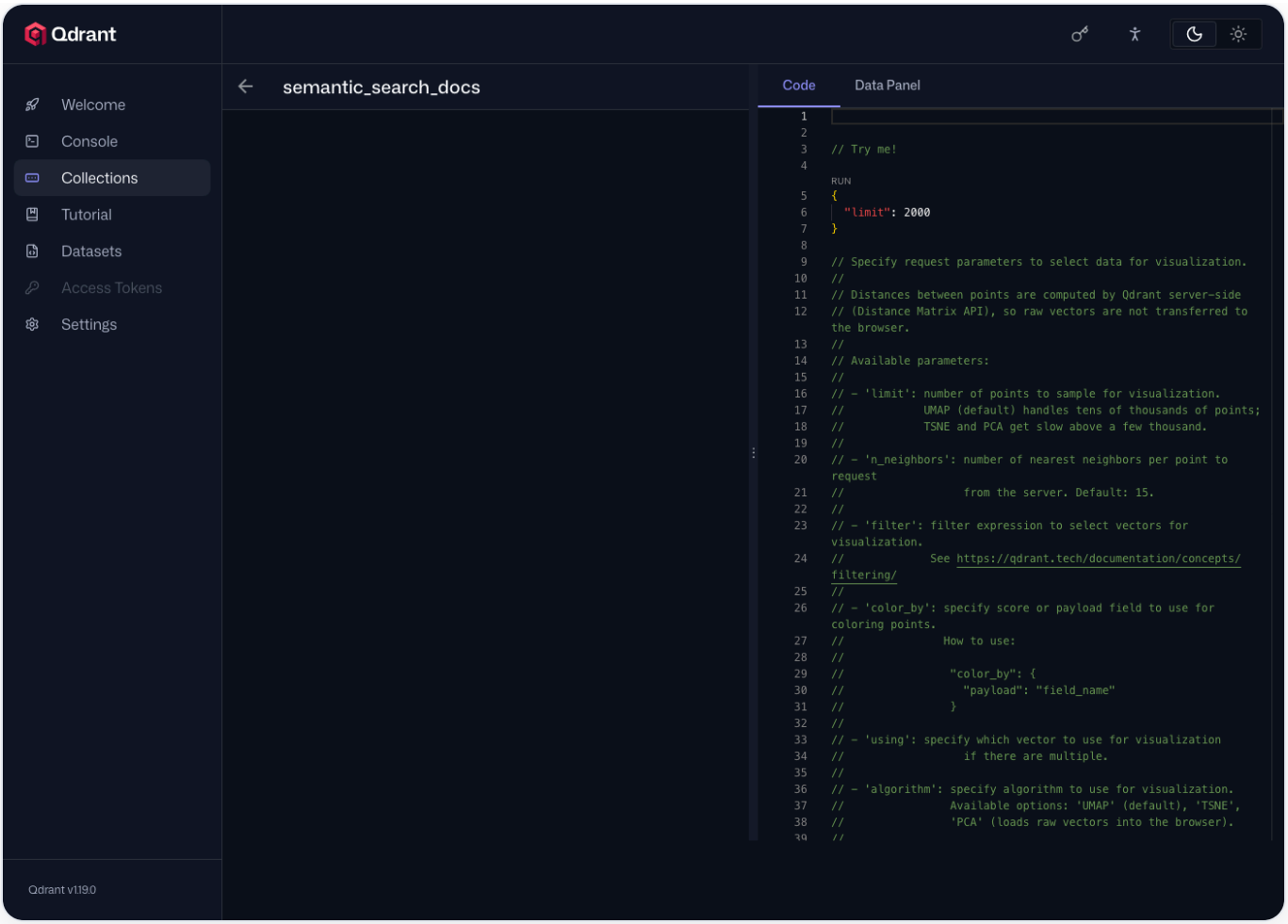
"What is an embedding?" retrieves the *Embeddings* paragraph at score 0.71; "How does the database make search fast?" surfaces the passages on nearest-vector return and the HNSW graph index — correct passage-level retrieval from an unstructured document.

Dashboard: Console & Visualize

The dashboard's **Console** runs raw REST queries against the collections, and the **Visualize** tab projects stored vectors to 2D server-side.



Dashboard Console — run REST API calls (list collections, get info, scroll points with filters) directly against Qdrant.



Dashboard Visualize tab — configurable UMAP/TSNE/PCA projection of the stored vectors.

Learning outcome

This project walks the full pipeline: **text representation** → **embedding generation** → **vector storage** → **similarity search**. Text is turned into 384-dim Sentence-Transformer embeddings, stored with metadata in a cosine-distance Qdrant collection, and retrieved by meaning. We demonstrated that keyword-free queries still return the right documents, that structured metadata filters compose with vector search, and that the same pipeline makes an arbitrary PDF semantically searchable.

embeddings **vector DB** **cosine similarity** **Top-K retrieval** **metadata filtering** **PDF RAG**